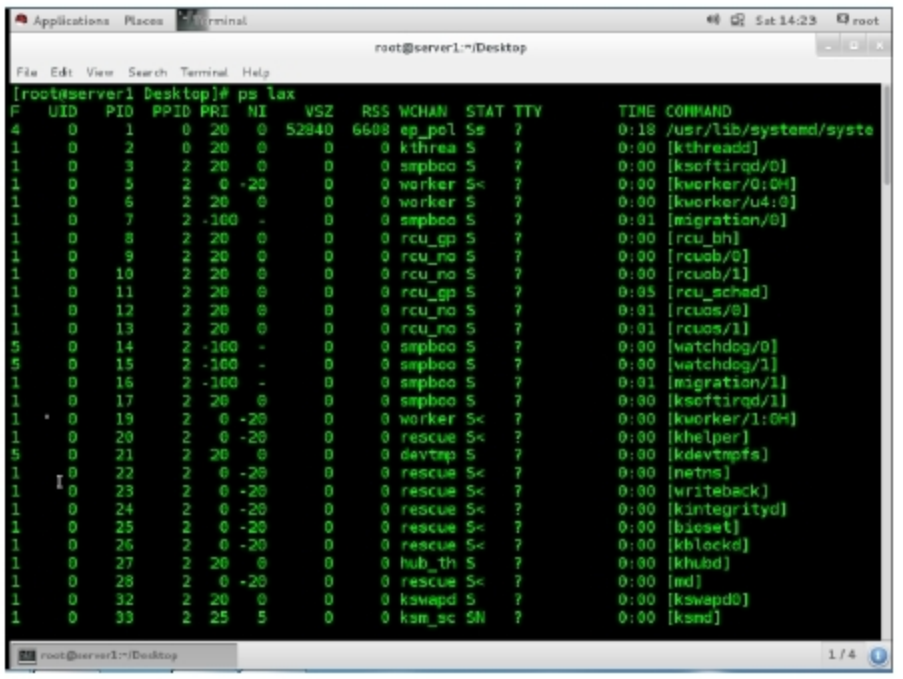

Figure 2: The output of *ps lax*

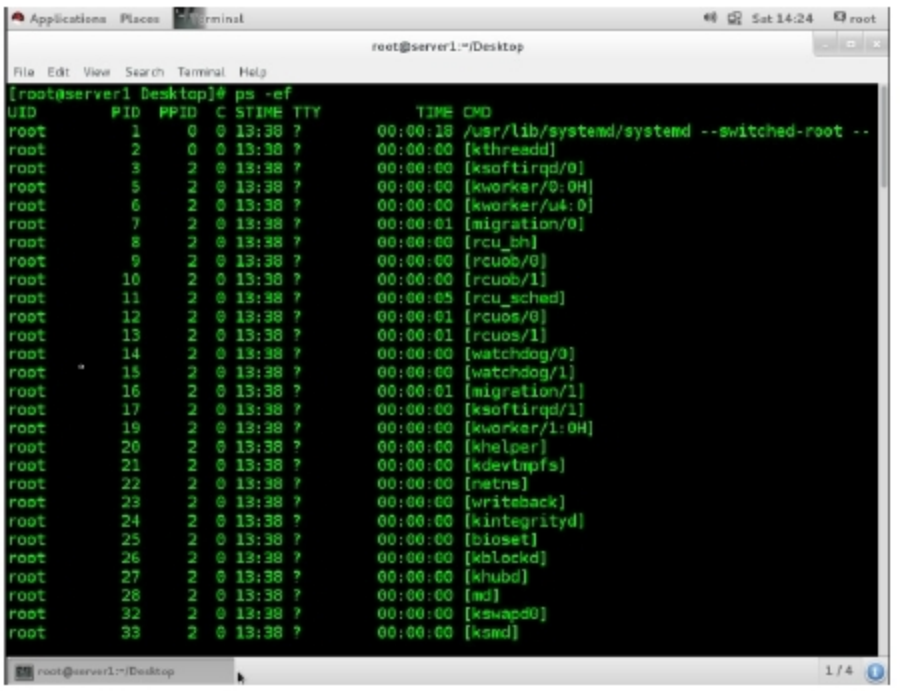

Figure 3: The output of *ps –ef*

- *ps* can be displayed in tree format to view parent/child relationships.
- The default output is unsorted. The display order matches that of the system process table, which reuses table rows as processes die and new ones are created. The output may appear chronological, but this is not guaranteed unless explicitly stated *–O, --sort* options are used.

Controlling jobs

Jobs and sessions: Job control is a command shell feature allowing a single shell instance to run and manage multiple commands. Without job control, a parent shell forks a child process to run a command, sleeping until the child process exits. When the shell prompt redisplays, the parent shell returns. With job control, commands can be selectively suspended, resumed and run asynchronously, allowing the shell to return for additional commands while the child process can run.

A foreground process is a command running in a terminal window. The terminal device ID (tty) is the process's controlling terminal. Foreground processes receive keyboard-generated input and signals, and are allowed to read from or write to the terminal (e.g., via *stdin* and *stdout*).

A process session is created when a terminal or console first opens (e.g., at login or by invoking a new terminal instance). All processes (e.g., the first command shell, its children and pipelines) initiated from that terminal share the same session ID. Within that session, only one process can be in the foreground at a time.

A background process is started without a controlling terminal because it has no need for terminal interaction. In a *ps* listing, such processes (e.g., services, daemons and kernel process threads) display a question mark (?) in the TTY column, while background processes that (improperly) attempt to read from or write to the terminal may be suspended.

Running jobs in the background

Any command can be started in the background by appending an ampersand (&) to the command line. The bash shell displays a job number (unique to the session) and the PID of the new child process. The command shell does not wait for the child process and redisplays the shell prompt.

The bash command shell tracks jobs, per session, in a table displayed with the *jobs* command as shown in Figure 4.

Background jobs can reconnect to the controlling terminal by being brought to the foreground using the *fg* command with the job ID (*%job number*).

The example *sleep* command is now running on the controlling terminal. The command shell is again asleep, waiting for this child process to exit. To resend it to the background, or to send any command in which the trailing ampersand was not originally included, send the keyboard generated suspend request (*Ctrl-z*) to the process. The suspend request takes effect immediately. The job is placed in the background. The pending output and keyboard typeahead are discarded.

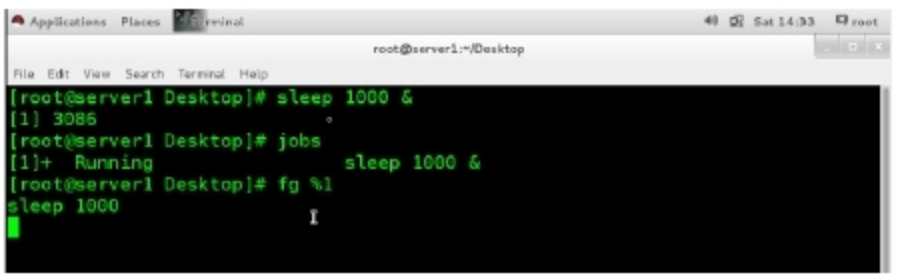

Figure 4: Process is sent to the background and then runs on the control terminal

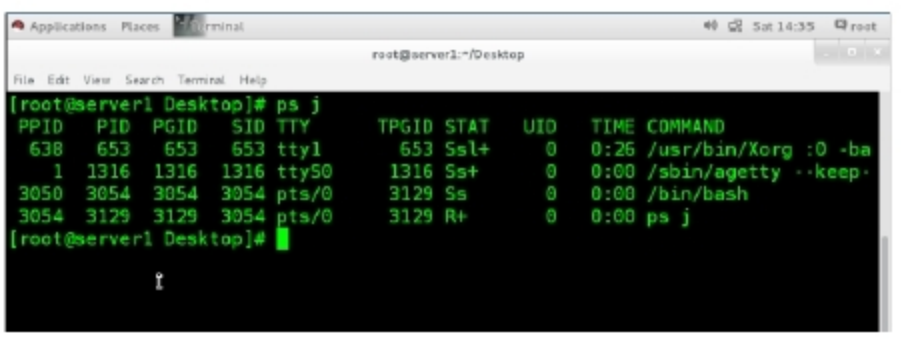

Figure 5: The job display output